

The Unofficial Scott Adams Specification v1.0

Table of Contents

Preface	2
Chapter 1: Story Files	3
Chapter 2: Header	6
Chapter 3: Actions	9
Chapter 4: Conditions	14
Chapter 5: Opcodes	17
Chapter 6: Vocabulary	26
Chapter 7: Rooms	29
Chapter 8: Messages	30
Chapter 9: Items	34
Chapter 10: Comments	36
Chapter 11: Trailer	37
Chapter 12: Game Loop	38
Chapter 13: Screen Model	43
Chapter 14: Known Games	45

Web version

This document is also available on the web: https://andwj.gitlab.io/scott_specs/

Preface

Introduction

This specification deals with the data file format used for the early adventure games by Scott Adams, as well as several games by Brian Howarth. The same format was used by a few other authors too.

This text-based file format has no commonly used short name, it is usually just called the "Scott Adams format". The terms **SACA**, standing for "Scott Adams Classic Adventure" is occasionally used, as well as **SAGA** for "Scott Adams Graphical Adventure". The original format was developed on the TRS-80 home computer, and appeared on several other systems too, including PCs running DOS.

Some later games (only four it seems) used an extended format called **SAGA+**, which included a much better parser which could understand more than two word phrases. That format is **not** covered by this specification.

There were also quite a few games released for the TI99/4A home computer which used a binary format. This format has a lot in common with the text format, although actions are encoded (and executed) quite differently. That format is described in a companion specification: https://andwj.gitlab.io/ti99_specs/

Acknowledgements

Bruce Hanson reversed engineered this format very early, and created the **ADVEDIT** editor program and his own interpreter. This software came with an excellent manual too, explaining most aspects of the TRS-80 file format.

Alan Cox created the **ScottFree** interpreter around a decade later, also reverse engineering the format (with help from Paul Doherty). He documented the format (quite tersely) in a file called **Definition**. This specification is basically an expanded version of that document.

Robert Schneck created the **ASFD** (Another ScottFree Driver) interpreter, based on the ScottFree code but largely rewritten, and he also clarified some aspects of the file format.

Mike Taylor created **ScottKit** which is an interpreter, a compiler and a de-compiler for Scott Adams story files. A lot of the information in this specification is based on his documentation.

Andrew Apted wrote this specification, though "compiled" may be a better term since most of the information herein is based on the work of others (those mentioned above).

Copyright

This document is under the **CC0** (Creative Commons Zero) license. This license is very similar to being **public domain**, however some jurisdictions do not recognise public domain as valid, hence we use the **CC0** license instead.

See here for more information: <https://creativecommons.org/publicdomain/zero/1.0/>

Chapter 1: Story Files

1.1 Low Level Encoding

Story files are plain text, in the ASCII character set.

They consist of a sequence of values, where each value is either a number or a string.

The following grammar uses an extended BNF (Backus-Naur Form), where `|` represents a choice, things in `[]` are optional, and things in `{ }` may be repeated zero or more times.

```
file      := { { whitespace } value }.  
value     := number | string.  
number    := ['-'] digit { digit }.  
digit     := '0' | '1' | '2' | ... | '7' | '8' | '9'.  
string    := '"' { char } '"'.  
char      := any ASCII character except '"'.  
whitespace := any ASCII code between 0 to 32.  
newline   := LF | CR | CR LF | LF CR.
```

Numbers are signed 16-bit integers. Note that two numbers in a row **must** be separated by whitespace. There is no support for other bases (such as hexadecimal), and no floating point values.

The number 65535 occurs in some story files, but is equivalent to -1. Interpreters must accept numbers between 32768 and 65535, but subtract 65536 from them to give a negative value.

Strings begin with a double quote (`"`), and are terminated by the same character. There is no general escape mechanism (such as the backslash used in many programming languages). However the back-quote (```) must be converted to a double quote, either at load time or when the string is displayed.

Strings can embed one or more line breaks by spanning multiple lines in the story file. Newlines in a file can be four possible things: a single LF (ASCII code 10), a single CR (ASCII code 13), a CR LF pair, or a LF CR pair. Interpreters **must** handle all four possibilities. It is recommended compilers output CR LF for all line endings.

FYI : The earliest games typically used a newline to separate each value, except (strangely) for the items, which had the item's name and location on a single line separated by a space.

1.2 High Level Structure

Story files are arranged as several chunks, where each chunk defines a particular type of element (such as rooms). These chunks must occur in a fixed order. There is nothing delimiting the start or end of these chunks - their size is either fixed (like the Header and Trailer chunks) or depends on a value given in the header.

The following table summarizes all the chunks occurring in a story file:

HEADER	Chapter 2
ACTIONS	Chapter 3
VOCABULARY	Chapter 6
ROOMS	Chapter 7
MESSAGES	Chapter 8
ITEMS	Chapter 9
COMMENTS	Chapter 10
TRAILER	Chapter 11

1.3 Identification

By Filename

There is no official file extension for story files.

The most common file extension is `.dat` or `.DAT`, but this is a very common and generic extension and is not adequate to distinguish a Scott Adams game from other types of adventures (e.g. Level 9 games often use this extension), or indeed to distinguish it from a myriad of other usages in the software world.

This specification recommends that the following file extensions are used to refer to (and detect) Scott Adams story files:

<code>.saga</code>	<code>.SAGA</code>
<code>.saca</code>	<code>.SACA</code>
<code>.sag</code>	<code>.SAG</code>
<code>.sac</code>	<code>.SAC</code>
<code>.sao</code>	<code>.SAO</code>

By Contents

There is no special sequence of characters (no "magic number") within the file which uniquely identifies a file as a Scott Adams game.

The fact that the file format is plain text (containing no zero bytes, for example) should be enough to distinguish it from many other types of adventure game code (like Z-code).

Chapter 2: Header

2.1 Format

The header consists of 12 numeric values at the very beginning of a story file.

The following table is a summary of each value. Note that the index shown in the first column is the **ordinal** position (1 to 12). Most of these values are self-explanatory, other ones are described in more detail below.

1	String storage
2	Last item number
3	Last action number
4	Last vocabulary pair
5	Last room number
6	Carry limit
7	Starting room
8	Total treasures
9	Word length
10	Lamp time
11	Last message number
12	Treasure room

2.2 Counts

Items, rooms, actions, messages (etc) all begin at zero. The header does not specify the total number for these, instead it specifies the last index. Hence you need to add one to get the total number.

2.3 String Storage

The first value is the number of bytes required to contain all the strings in the story file.

Early interpreters required this to ensure they had enough memory. Modern Interpreters can completely ignore this number.

Compilers should either compute it, or leave it as zero.

Calculation Method

Firstly, consider the vocabulary words (verbs and nouns). Each word requires a fixed amount of space: the word length in the header + 1. Short words get padded out, long words get truncated:

```
vocab_size = num_vocab * 2 * (word_length + 1)
```

Secondly, each string requires one byte to terminate it. Embedded newlines also only occupy a single byte. So take all the strings of each room, item and message (even room #0 and message #0) and count the total number of bytes (including the terminating byte).

```
normal_size = sum of item + room + message strings
```

Thirdly, consider item names which contain a vocabulary word within forward slashes (/). These seem to need an extra byte, perhaps because the part in slashes is allocated as a separate string (while **keeping** the slashes too!).

```
slashed_items = number of items with a vocab word
```

Lastly, there seems to be one extra byte. Putting it all together we get:

```
string_storage = vocab_size + normal_size + slashed_items + 1
```

2.4 Vocabulary Pairs

The vocabulary is a list of words, with verbs and nouns interleaved. When the number of verbs is not the same as the number of nouns, the smaller list is padded with empty strings or invalid words.

See the [Vocabulary](#) section for more information about how verbs and nouns are handled.

2.5 Total Treasures

The number of treasures required to win the game. This can be less than the actual number of treasure items in the game, in which case the player does not need to place them all in the treasure room to win.

This value **must** not be zero, because that can cause existing interpreters to crash when a player uses the **SCORE** command. Compilers should detect when a story file contains no treasures and store **1** here.

2.6 Word Length

The word length is how many letters of a word are significant when comparing them. Only 3, 4 and 5 have been seen in existing games. Interpreters should honor any value of 2 or more.

2.7 Lamp Time

This is how long (the number of turns) the lamp will last when lit. When it runs out, you will not be able to see when it is dark, and the story will probably cause bad things to happen to you.

The special value `-1` can be used to inhibit the lamp counter. This is a feature introduced for later games, such as *Gremlins*. Some early games just use a very high value, such as `15000` and `32767`. It is recommended that values `>= 9999` are converted to `-1` and hence also inhibit the lamp counter.

See the [Special Items](#) section for more information about the lamp.

2.8 Treasure Room

The room number (starting at zero) where the player must place all the treasures to win the game. The player must also use the `SCORE` command to trigger a check that all the treasures have been placed there.

Chapter 3: Actions

3.1 Format

The number of actions is specified by the header chunk.

Each action consists of 8 numeric values. However these numbers use a packing scheme, where each number actually encodes two things (by simply multiplying one by a constant and adding the other).

Here is a summary of the eight numbers:

1	Verb	Noun
2	Condition 1	Arg 1
3	Condition 2	Arg 2
4	Condition 3	Arg 3
5	Condition 4	Arg 4
6	Condition 5	Arg 5
7	Opcode 1	Opcode 2
8	Opcode 3	Opcode 4

3.2 Terminology

Different tools (and their documentation) use different terms for what is called an "opcode" here. For example, **ADVEDIT** calls them a "command", **ScottKit** calls them a "result", and **ScottCom** calls them an "operation".

3.3 Verb/Noun Pair

The first number is a Verb/Noun pair, encoded as $(150 * \text{verb} + \text{noun})$. The noun may be #0 (**ANY**) for actions which do not need a noun, like **INVENTORY** and **SWIM**, or when the game is checking something and it does not matter what the noun is. For example: a **CHOP ANY** action could be used to make sure the player is holding an axe.

When the verb is zero, this is an **implicit** action, otherwise it is an **explicit** action. Implicit actions are run between turns to implement things happening in the world, like an NPC moving around the map. Explicit actions are only executed in response to the player's input.

In the original games, the implicit actions came first in the story file, before all the explicit actions, so the interpreter could stop processing them once reaching an explicit action. Modern interpreters like **ScottFree** scan the whole action list to execute implicit actions, and some games built with **ScottKit** require this behavior.

See [Chapter 12](#) has some more information about how (and when) actions get executed.

3.4 Conditions

Each action has 5 conditions which must be satisfied before the action can be executed. The conditions are encoded as $(\text{condition} + 20 * \text{argument})$. Most conditions require an argument, but there are a few which do not.

Condition #0 is rather special since it does not check anything, rather it simply appends its argument to a list which provides the parameters to the opcode(s) of the action.

See [Chapter 4](#) for more information about all the available conditions.

3.5 Opcodes

Each action has 4 opcodes. These are encoded in the last two numbers as $(150 * \text{opcode1} + \text{opcode2})$ and $(150 * \text{opcode3} + \text{opcode4})$.

Opcode #0 is a no-op (it does nothing). This is used to pad the slots when the action does not need to use them all.

Some opcodes require a parameter (or even two). Condition #0 is used to populate a small parameter list.

See [Chapter 5](#) for more information about all the available opcodes.

3.6 Executing an Action

We say an action is **matched** when its verb matches the input verb, its noun matches the input noun, and all its conditions are satisfied. For implicit actions, the verb must be zero, and the noun is actually a value between 1 and 100 giving the random chance of executing the action in the current turn.

When an explicit action has a noun of zero (**ANY**), then the action matches any input noun, as well as when the player's input did not provide a noun.

Actions are only executed once they are matched. Execution is mainly a matter of iterating through the opcodes and executing each one. However the **continue** and **get** opcodes can affect execution, see the following sections for more information.

Parameters

Some opcodes require a parameter, and a few require two.

These parameters are stored in the conditions, using condition #0. Collecting these parameters can be done at load-time (recommended) or at run-time. They conceptually form a list which can contain up to five values.

When an opcode needs a parameter, it grabs the **next** one in the list. An interpreter needs a variable to keep track of which parameter is the current one. What happens when the parameter list runs out is implementation-defined.

3.7 Continued Actions

Opcode #73 is **continue** — a special opcode which permits more complex actions to be constructed.

When an interpreter executes the **continue** opcode, it must set a local flag, and then finish executing any remaining opcodes in the current action. Once finished (and it does not matter if the action was aborted or not), the interpreter checks the local flag, and when set it must attempt to execute **ALL** subsequent actions which have a Verb/Noun pair of zero.

These actions are the continuation of the initial action. Like normal actions, their conditions **must** be satisfied before the opcodes are executed. There may be multiple continued actions: they are terminated by a non-zero Verb/Noun pair (which is not executed) or the end of the action list.

Both **implicit** and **explicit** actions can use continued actions.

Example

The following example is from *Lone Survivor* by Brian Howarth.

There are three actions together. The first action has a condition checking the player is wearing the radiation suit — none of these actions is executed unless that condition is met. When satisfied, the **continue** opcode causes the next two actions to be attempted. The second action is only executed when the oxygen counter has gone below zero. The third action has no conditions, so is always executed, and puts the decreased counter value back into counter 4.

```
occur 100%
  when carried RAD_SUIT
    swap_counter 4
    dec_counter
    continue

occur 0%
  when counter_le 0
    print "Suit oxygen"
    print "is exhausted"
    print "I'm asphyxiated!"
    die

occur 0%
  # always
  swap_counter 4
```

3.8 Terminated Actions

The **get** opcode causes the current action to be **terminated** when the player is at their carry limit. This simply means that any remaining opcodes are skipped (not executed).

It is not clear whether continued actions should still be executed after a failed **get**. This combination occurs rarely, and the cases which do occur don't provide any clue either way. Hence the behavior is currently implementation-defined.

NOTE : the **ScottFree** interpreter never terminates an action when **get** fails, but this is incorrect and leads to odd messages (at best), or breaking a game (at worst).

Buggy Games

It's very unfortunate, but some games, even commercial offerings, use the `get` opcode in a way that can break a game — cause it to be unwinnable — simply because they don't take into account the possibility that the player was at their carry limit at that point in the game.

This is a dilemma for interpreters. While it would be reasonable for an interpreter to minimize the risk of games breaking by making the `get` opcode unconditionally give the item to the player, that often leads to situations where the player is carrying more than the normal limit, sometimes several items more.

A possible solution to this dilemma is to make the `get` always work, but when this exceeds the inventory limit attempt to "fumble" another item out of the inventory. Such an item should (a) have a noun, so the player can pick it up again, and (b) have no custom action for dropping the item, like the mirror in *Adventureland*.

Chapter 4: Conditions

4.1 Summary

The following table summarizes the available conditions.

Most of the conditions can be negated, requiring the opposite to be true, and this document treats them as a single concept. The table below shows the normal ID number in the first column, and the negated ID number in the second column. Most condition also take an argument.

ID	NEGATED	NAME	ARGUMENT
0	-	param	-
1	6	carried	Item
2	5	here	Item
3	12	present	Item
4	7	at	Room
8	9	flag	Flag
10	11	loaded	-
13	14	exists	Item
18	17	moved	Item
15	-	counter_le	Number
16	-	counter_gt	Number
19	-	counter_eq	Number

The condition names used here are from the **ScottKit** compiler.

Due to the way actions are encoded, there can only be 20 different conditions, and all of them are currently used — no possibility of adding new ones in the future!

4.2 Detailed Listing

param | id=0

This is not a real condition, rather it just appends its argument onto a small list which serves as the parameters of the opcodes (if or when they are executed).

carried | id=1 | negated=6

Checks whether the player is carrying a particular item.

here | id=2 | negated=5

Checks whether a particular item is in the same room as the player.

present | id=3 | negated=12

Checks whether a particular item is either carried by the player, or in the same room as the player.

at | id=4 | negated=7

Checks whether the player is in a particular room.

flag | id=8 | negated=9

Checks whether a particular flag is set.

The argument **must** be in the range 0 .. 31.

loaded | id=10 | negated=11

Checks whether the player is carrying at least one item.

The argument is not used for this condition.

exists | id=13 | negated=14

Checks whether a particular item is in the game, i.e. its location is anything except room #0.

`moved` | id=18 | negated=17

Checks whether a particular item is in a different location than its start location.

`counter_le` | id=15

Checks whether the main counter variable is less than or equal to the argument.

`counter_gt` | id=16

Checks whether the main counter variable is greater than the argument.

Technically, `counter_gt` is just the negated form of `counter_le`. However it makes the source code of a game (in ScottKit syntax) easier to read and understand to treat them separately.

`counter_eq` | id=19

Checks whether the main counter variable is equal to the argument.

NOTE : the comment "Only seen in Brian Howarth games so far" in the ScottFree source code is misleading — this opcode is used in several early games by Scott Adams, such as *The Count* and *Strange Odyssey*.

Chapter 5: Opcodes

5.1 Summary

The following table summarizes the available opcodes.

Some opcodes take a parameter, and a handful take two parameters. These parameters are set by condition #0 — see the previous chapters for more information).

ID	NAME	PARAM 1	PARAM 2
0	<code>nop</code>	-	-
52	<code>get</code>	Item	-
53	<code>drop</code>	Item	-
54	<code>goto</code>	Room	-
55	<code>destroy</code>	Item	-
56	<code>set_dark</code>	-	-
57	<code>clear_dark</code>	-	-
58	<code>set_flag</code>	Flag	-
59	<code>destroy2</code>	Item	-
60	<code>clear_flag</code>	Flag	-
61	<code>die</code>	-	-
62	<code>put</code>	Item	Room
63	<code>game_over</code>	-	-
64	<code>look</code>	-	-
65	<code>score</code>	-	-
66	<code>inventory</code>	-	-
67	<code>set_flag0</code>	-	-
68	<code>clear_flag0</code>	-	-
69	<code>refill_lamp</code>	-	-
70	<code>clear</code>	-	-

71	save_game	-	-
72	swap	Item	Item
73	continue	-	-
74	superget	Item	-
75	put_with	Item	Item
76	look2	-	-
77	dec_counter	-	-
78	print_counter	-	-
79	set_counter	Number	-
80	swap_room	-	-
81	swap_counter	Counter	-
82	add_to_counter	Number	-
83	subtract_from_counter	Number	-
84	print_noun	-	-
85	println_noun	-	-
86	println	-	-
87	swap_specific_room	Room	-
88	pause	-	-
89	draw	Picture	-

Notes

The opcode names used here are mainly from the **ScottKit** compiler.

Opcodes #1 to #51 are not listed — they just display messages #1 to #51.

Opcodes #102 and above are also absent — they just display messages #52 onward.

Opcodes #90 to #101 are not used by any known game. Interpreters **must** detect unknown opcodes when loading the story file and reject it (show an error and quit), since the game will probably not function correctly by simply ignoring the unknown opcodes.

FYI : the original version of *Adventureland* was written in BASIC and only used opcodes #1 to #72 — the remaining opcodes were added later.

5.2 Detailed Listing

nop | id=0

The opcode does nothing at all. It is used to pad out the set of opcodes in an action when less than four is needed.

get | id=52

Attempts to move a particular item (given by the parameter) to the player's inventory.

When the player is under their carry limit, then the item is moved successfully. No message is displayed.

Otherwise, a refusal message should be displayed, and the current action should be **terminated**. See the [Terminated Actions](#) section in Chapter 3 for more information on what this means.

drop | id=53

Moves a particular item (given by the parameter) to the same location as the player.

goto | id=54

Moves the player to a particular room (given by the parameter), and updates the screen.

destroy | id=55

Moves a particular item (given by the parameter) out of the game, i.e. to room #0.

`set_dark` | id=56

Sets flag #15 (the standard DARK flag).

`clear_dark` | id=57

Clears flag #15 (the standard DARK flag).

`set_flag` | id=58

Sets a particular flag (given by the parameter).

The parameter **must** be in the range 0 .. 31.

`destroy2` | id=59

Equivalent to the `destroy` opcode (id=55).

OPEN QUESTION : whether or not this opcode did something different in the original interpreters. In the BASIC version of *Adventureland* they are equivalent—they both GOTO the same line number.

`clear_flag` | id=60

Clears a particular flag (given by the parameter).

The parameter **must** be in the range 0 .. 31.

`die` | id=61

Displays a message that the player has died, then moves the player to the very last room, clears flag #15 (the DARK flag), then updates the screen by performing the `look` opcode (id=64).

The very last room is called the "Limbo" room. It often gives the player a chance to get back to the normal world. The player does not lose any items (or any points) via this opcode.

`put` | id=62

Moves a particular item (given by the first parameter) to a particular room (given by the second parameter).

`game_over` | id=63

Displays a message that the game is over, waits 5 seconds, then quits.

`look` | id=64

Updates the screen to show where the player is, what the obvious exits are, and what items are visible.

When flag #15 (the DARK flag) is set, and there is no light source nearby, then it instead shows a message saying it is too dark to see, and does **not** show the obvious exits or any items.

NOTE : numerous games, especially ones by Brian Howarth, do not execute this opcode when the player types the `LOOK` command with no noun. Interpreters which support a basic screen model (i.e. not the usual split-screen model) cannot rely on this opcode. See the [Single Window](#) section in Chapter 13 for more information.

`score` | id=65

Displays the number of treasures the player has stored, as well as a score value (the percentage of treasures stored, between 0 and 100).

When the number of treasures stored is $\geq$ `Total treasures` field in the header, then this displays a congratulatory message and quits.

`inventory` | id=66

Displays what the player is carrying.

`set_flag0` | id=67

Sets flag #0.

`clear_flag0` | id=68

Clears flag #0.

`refill_lamp` | id=69

Resets the remaining lamp time to its original value, moves the lamp item to the player's inventory, and clears flag #16 (the standard `LIGHT_OUT` flag).

`clear` | id=70

Clears the screen.

NOTE : this gets used a lot in the original games, often before the `die` opcode, and probably just cleared the lower window where commands are entered. For interpreters with a basic screen model, it may be better to only display a horizontal ruler, or even nothing at all.

`save_game` | id=71

Saves the current game. The interpreter is responsible for asking for a filename, writing the state of play into that file, and displaying a message about the success or failure of the save.

`swap` | id=72

Swaps the location of the two items given by the parameters.

This is often used to "turn on" or "turn off" the lamp, by swapping the real lamp (item #9) with an different, unlit version of it.

`continue` | id=73

Sets an internal flag which causes one or more following actions to be executed (as long as their conditions allow it).

This can be used in both implicit and explicit actions.

See the [Continued Actions](#) section in Chapter 3 for more information about how this opcode works.

`superget` | id=74

Unconditionally gives a particular item (given by the parameter) to the player. None of the normal checks, such as the carry limit, are performed. No message is displayed.

`put_with` | id=75

Moves a particular item (given by the first parameter) to the same location as another item (given by the second parameter).

The location of the second item is allowed to be `nowhere` (room #0), or the player's inventory,

`look2` | id=76

Equivalent to the **look** opcode (id=64).

OPEN QUESTION : whether or not this opcode did something different in the original interpreters.

dec_counter | id=77

When the counter variable is ≥ 0 , subtracts one from it.

OPEN QUESTION : whether this should allow going below zero.

print_counter | id=78

Displays the value of the counter variable, in decimal.

set_counter | id=79

Directly sets the counter variable to a particular value (given by the parameter).

swap_room | id=80

Swaps the player location variable with the swap room variable.

The interpreter should set a flag, and **only** update the screen once it is time to get input from the player, or when the **pause** opcode is performed. This is needed since the original value of the swap room variable is room #0, so games often pair this opcode with a **goto** opcode to set things up, and they don't actually want to display the description or contents of room #0.

swap_counter | id=81

Swaps the value of the counter variable with a particular saved counter (given by the parameter).

There are 16 saved counters, numbered **0** .. **15**. These are separate from the main counter variable.

Story files typically use this in a normal action, then use continued actions to modify the saved counter and/or conditionally do things based on its value, then finish with a continued action which swaps the value back again.

NOTE : this opcode is called "select_counter" in ScottKit, however that name is quite misleading and doesn't fully capture what this opcode actually does.

add_to_counter | id=82

Adds a value (given by the parameter) to the selected counter. The final value should be clamped to the range -1 .. 32767.

`subtract_from_counter` | id=83

Subtracts a value (given by the parameter) to the selected counter. The final value should be clamped to the range -1 .. 32767.

OPEN QUESTION : whether this should allow going below zero.

`print_noun` | id=84

Displays the name of the noun used in the player's command.

Most commonly used with the `SAY` verb when saying magic words. An interpreter could display the word in an distinctive way, e.g. using an italic font or a special color.

`println_noun` | id=85

Displays the name of the noun used in the player's command, then does a `println` (opcode #86).

`println` | id=86

Moves the output cursor to the start of the next line on the screen.

`swap_specific_room` | id=87

Swaps the player location variable with one of the saved rooms (given by the parameter).

There are 16 saved rooms in addition to the main swap room variable. They are numbered 0 .. 15.

See the `swap_room` opcode for details about updating the screen afterwards.

`pause` | id=88

Pauses for 2 second.

OPEN QUESTION : sources differ on what the delay should be: 1 second is specified in the manual for ADVEDIT, 1.5 seconds is mentioned in ScottFree code as the time used by ZX Spectrum games, ScottFree itself uses 2 seconds, but the ASFD interpreter (a derivative of ScottFree) uses 1 second.

Show an alternate picture for a while.

Interpreters which support graphics **automatically** show a picture for the current room, where the room number determines which picture to show. Picture 0 is often used when it is too dark to see.

This opcode is used to select a different picture to show. The parameter (a picture number) is usually in a different set of images than the rooms, so 1 or 2 are not the same as the pictures for rooms #1 and #2.

Example 1: in *Robin of Sherwood*, a draw opcode is used to show an image of Herne the Hunter when the player encounters him early in the game.

Example 2: in *Gremlins*, the draw opcode is used when the player examines Gizmo or one of the gremlins.

Chapter 6: Vocabulary

6.1 Format

The vocabulary is a list of words, with verbs and nouns interleaved. Each word is a string value.

The number of verb/noun pairs is specified in the header chunk. When the number of verbs is not the same as the number of nouns, the smaller list is padded with empty strings or invalid words.

Only the first few letters of a verb and noun is significant. How many is specified by the **Word Length** field of the header, which is usually between 3 and 5. Verbs and nouns in the vocabulary list are allowed to be longer than this.

6.2 Limit on Words

Due to the way actions are encoded, there are hard limits on the number of verbs and nouns which can be used in a story file, as follows:

- only **149** nouns can be used by actions. That excludes the dummy noun #0, but includes the six standard directions. Additional nouns may exist, and could be used for the short name of items (but only ones which never appear in an action).
- only **217** verbs can be used by actions. That excludes the dummy verb #0, but includes the three standard verbs. This is a consequence of dividing the highest 16-bit signed integer **32767** by **150**.
- verb #218 is technically **possible**, but it cannot be used with the full range of nouns, and hence it is not desirable that a compiler accept a story file with more than 217 verbs.

These limits include synonyms, i.e. each synonym uses up a slot in the noun or verb table.

Most of the commercially published games use less than **130** vocabulary words.

6.3 Valid Words

Words can consist of letters, digits and symbols. Words in a story file may be given in uppercase, lowercase or some combination of the two. They must be matched in a case-insensitive method, for example by converting them to uppercase at load time.

NOTE : these rules also apply to the short name of items.

Symbols

The following symbols are used in existing games and hence **must** be supported by interpreters: hyphen (-), dollar sign (\$).

It is recommended to also allow these symbols: apostrophe ('), plus sign (+), at sign (@), percent sign (%), hash sign (#).

Some games contain nouns containing a full-stop (.). This could be problematic for interpreters which allow multiple commands to be given on a single line. The only known usage are the words **DR.** and **DR.S** — an interpreter could handle these as a special case when parsing the user's input. For interpreters which do not allow multiple commands, the full-stop can be accepted as a regular symbol.

Compilers should produce a warning or error when any symbol character other than those mentioned above are used in a vocabular word. In particular, a forward slash (/) is **never** valid in the short name of an item and must produce an error.

Invalid Words

Some story files contain invalid words. Typically these are an empty string, a string of nothing but spaces, or a single full-stop (.). They can also occur in synonyms (see below).

These are probably just slots which ended up not being used in the final game. Interpreters should detect which words are invalid and ensure that they are never matched against user input.

6.4 Synonyms

A verb or noun which begins with an asterix (*) is a synonym for the previous verb or noun. There can be multiple synonyms for a word.

When an interpreter finds a synonym, it should use the index of the original word (the last one without an asterix). Note that invalid words can exist amongst the group of synonyms of a word, for example *. appears in the *Adventureland* vocabulary.

6.5 Standard Verbs

The following verbs are the same in every game. Interpreters can assume that **GET** (for example) is always verb #10. Compilers must ensure that these verbs exist and occupy their corresponding slot.

0	AUTO
1	GO
10	GET
18	DROP

Note that verb #0 is a dummy, it should never be matched with user input.

6.6 Standard Nouns

The following nouns are the same in every game. Interpreters can assume that **SOUTH** (for example) is always noun #2. Compilers must ensure that these nouns exist and occupy their corresponding slot.

0	ANY
1	NORTH
2	SOUTH
3	EAST
4	WEST
5	UP
6	DOWN

Note that noun #0 is a dummy, it should never be matched with user input.

6.7 Abbreviations

The story file is not able to specify synonyms for the six standard directions, so the interpreter is responsible for converting the single letter versions (**N, S, E, W, U, D**) to their corresponding noun.

An interpreter may also convert other single letter verbs to their full form, provided that the single letter is not already defined as a verb in the story file. Recommended conversions are:

I	INVENTORY
L	LOOK
X	EXAMINE
Z	WAIT

Chapter 7: Rooms

7.1 Format

The number of rooms is specified by the header chunk.

Each room consists of six number values followed by a single string. The string is the description of the room, and the numbers are the locations where the six standard directions will take the player.

The order of direction numbers in the room definition matches the six standard direction nouns: **NORTH**, **SOUTH**, **EAST**, **WEST**, **UP** and **DOWN**.

7.2 Descriptions

Room descriptions may begin with an asterix (*), which means the description should be displayed as-is (but without the asterix).

When the asterix is absent, the interpreter needs to prefix the description with a message like "I am in a " (or the equivalent for second person games).

7.3 Room #0

Room #0 is a dummy room, it represents **nowhere**. Despite the fact that the player normally never enters this room, it still has an entry in the story file.

Items can be moved to room #0 to "destroy" them, or begin in room #0 when they need to be brought into the game at a later time. When an exit in a room cannot be used, it specifies room #0 as the destination.

Chapter 8: Messages

8.1 Format

The number of messages is specified by the header chunk.

Each message is a single string value.

Back-quote (``) characters in a message string must be converted to double quote (") characters by the interpreter, either at load time or while printing the message. Message strings may also contain line breaks, by simply extending into the next line of the file.

8.2 Usage

Messages are printed to the screen by opcodes in the range 1 .. 51 and 102 .. 149. The latter range of opcodes subtracts 50 to produce the message number in the range 52 .. 99.

Message #0 is a dummy and is never used. It should be empty.

8.3 Extra Messages

Normally only 99 messages can be used in a story file, and no commercially published game uses any more than this.

However.... messages #100 to #167 are technically **possible** by using opcodes 150 .. 217. The catch is that they can only be used in the first or third opcode of an action, never in the second or fourth opcode of an action.

This limitation makes them hard to support in a compiler — changes to the source code which seem innocent (from the user's perspective) could be the difference between their story file compiling or breaking. This specification recommends a compiler either not support extra messages, or add a new `print_extra` command which forces the message to be in the range 100 to 167, as distinct from the normal `print` command which forces the message to be in the range 1 to 99.

8.4 Display of Messages

Messages are printed to the lower window.

What happens when two or more messages are displayed by an action, or a sequence of continued actions, is not well-defined. Some interpreters printed each message on a line by itself, while others printed the messages on the same line, inserting a space between them. The TI99/4A games always did the latter.

Hence it is an open question whether an interpreter should always print messages on a new line, or always print them on the same line, or some combination of the two.

8.5 Hard-coded Messages

There are numerous messages which are hard-coded in the interpreter, and story files have no mechanism to change them.

Most games use the first person perspective, such as "I can't do that" and "I am dead". However, at least one game (*Robin Of Sherwood*) uses the second person perspective, with messages like "You see nothing special".

Interpreters should detect whether a game uses first person or second person perspective, and select a set of appropriate output messages. It can do this by examining all the messages, including room descriptions which begin with an asterix (*), and check whether it starts with the word **I** or **YOU**. Tally the results and pick the highest one.

List of Messages

The following tables show the hard-coded messages used in the BASIC version of *Adventureland*. Later games and interpreters often used different messages, and while interpreters are free to use their own messages, it is recommended to use ones similar to these to keep the same flavor of the original games.

Words inside angle brackets (<>) denote values which an interpreter must supply, such as the number of treasures. Adapting these messages for the second person perspective is left as an exercise for the reader.

Command Parsing

Tell me what to do?
You use word(s) I don't know!
I don't understand your command.
I can't do that yet.
OK

Room Descriptions

I can't see. It is too dark!
I'm in a <ROOM NAME>
Obvious exits:
Visible items here:

Inventory

I'm carrying:
Nothing!

Save and Restore

Save failed.
Load failed.
Saved.

Winning and Losing

I've stored <NUM> treasures. On a scale of 0 to 100, that rates a <NUM>.
Well done.
I am dead.
The game is now over.
Another game?

Moving Around

Give me a direction too.
Dangerous to move in the dark!
I fell down and broke my neck.
I can't go in that direction.

Get and Drop

What?
Nothing taken.
Nothing dropped.
It is dark.
It's beyond my power to do that.
I don't see it here.
I'm not carrying it!
I already have it.
I've too much to carry!

The Lamp

Light runs out in <NUM> turns.
Light has run out!

Chapter 9: Items

9.1 Format

The number of items is specified by the header chunk.

Each item is a pair of values: a string for its displayed name, followed by a number for its initial location (a room number).

Item #0 is a valid item.

Due to the way actions are encoded, there is a limit of 150 items.

9.2 Location

Items placed in room #0 are not in the game. Items can be brought into the game by moving them into a valid room, or taken out of the game by moving them to room #0.

The special values **255** and **-1** indicate that the item begins in the player's inventory.

9.3 Treasures

When the display name begins with an asterix (*), then the item is considered to be a treasure. The asterix remains part of the name, and is shown in room descriptions and inventory listings. There is usually another asterix at the end, but that is not required.

9.4 Vocabulary Word

Items which can be picked up by the player **must** have their vocabulary word encoded in their string. This is done by surrounding the word with forward slashes (/) and appending this to the end of the item's display name. Interpreters need to detect this, extract the word, and remove the whole thing from the display name.

The empty sequence (//) is allowed, but is equivalent to the item having no vocabulary word.

Interpreters must match against this vocabulary word when the user has typed the **GET** or **DROP** verb (or one of their synonyms) and the input is **not** handled by any existing action rule. Interpreters must first apply synonyms on the input noun before doing this match.

It is legal for multiple items to share the same vocabulary word. Games mainly do this for items which appear to change over time (but in reality they are swapped with another item). When multiple items are available for a **GET** or **DROP** command, the result is implementation-defined.

Items without a vocabulary word are not able to be picked up by **GET** or dropped by **DROP** except via an explicitly coded action. Most items lacking a vocabulary word are scenery or NPCs.

The vocabulary word inside `//` is usually also defined as a noun (in the vocabulary chunk of the story file), but does not have to be. When absent, the player will be unable to perform any action with that object except for getting it and dropping it. Compilers should always create a true noun for any item which can be picked up or dropped.

9.5 Special Items

The Lamp

Item #9 is usually a lamp or light source of some kind. When this item is in the game (i.e. its room number is not #0), then the **Lamp Timer** will count down. Once it reaches zero, and it is dark, then the player will not be able to see and the game may cause something bad to happen.

A story file typically implements "turning on" or "lighting" the lamp by having a separate object which represents the lamp when off, and then swapping the locations of the two lamps (usually the lit lamp will start `nowhere` in room #0).

It is the interpreter's responsibility to inform the player when their lamp is running low, and when it has completely run out. See the [Lamp Timer](#) section in chapter 12 for more information.

When the header specifies a lamp time of `-1`, then the lamp timer is not used, allowing item #9 to be used as an ordinary object.

Chapter 10: Comments

10.1 Format

For every action in the action chunk, there is a corresponding string value in this chunk.

10.2 Usage

These comments are only for game authors to keep notes about what the corresponding action is doing. They are purely informational and are not needed by an interpreter to play a story file.

Interpreters which want to read the trailer information will need to read each comment string. However interpreters are free to simply stop reading the story file after the items.

Chapter 11: Trailer

11.1 Format

The story file ends with three numeric values, which are:

- the version number
- the adventure number
- the checksum

There may be additional bytes (typically zero) after these numbers. Interpreters should stop reading the file once the checksum value has been read.

11.2 Version Number

This is a version number multiplied by 100, e.g. 678 represents version 6.78. It is purely informational and has no effect on the game, though an interpreter could potentially show it somewhere.

Compilers should allow it to be specified somehow, perhaps allowing it to be absent (with a default of 1, representing version 0.01).

11.3 Adventure Number

The adventure number is also purely informational.

The original games by Scott Adams begin at 1 and increase up to 14 (or so), though some of his games re-use the same number. The games by Brian Howarth (in his *Mysterious Adventures*) are numbered 1 to 11.

Interpreters can just ignore this number, but could potentially show it somewhere.

Compilers should allow it to be specified somehow, perhaps allowing it to be absent (with a default of 1, indicating the very first game by that particular author).

11.4 Checksum

The checksum is computed as: $(2 * \text{num_actions} + \text{num_items} + \text{version})$

Interpreters **should not** refuse to play a game when the checksum is wrong.

Compilers should either compute a valid checksum, or leave it as zero.

Chapter 12: Game Loop

12.1 The State of Play

The state of play consists of the following variables:

the player’s location
the remaining lamp time
a counter variable
a swap room variable
a set of 32 flags
a set of 16 saved counters
a set of 16 saved rooms
the location of every item

The header chunk specifies the initial value for the player’s location, the remaining lamp time. The items chunk specifies the initial location of all the items in the game.

All the other variables must have an initial value of zero.

Known Flags

The meaning of most flags is not fixed, each story file can use them for whatever purpose they want.

However two flags have a fixed meaning, and are tested and modified directly by an interpreter. These are shown in the following table:

15	DARK	It is dark, the player cannot see.
16	LIGHT_OUT	The lamp has run out.

12.2 Playing a Game

The following pseudo-code is overly simplistic, but gives a general idea on what needs to happen to play a game:

```
set up the screen.  
initialize the state-of-play variables.  
  
loop indefinitely  
{  
    execute all implicit actions.  
    read a line of input from the user.  
    parse the input into a verb/noun pair.  
    execute the matching explicit action.  
    handle the lamp timer.  
}
```

12.3 Implicit Actions

Implicit actions are what allow things to happen in the game world which are independent of what the player does.

It is important that they are run **before** the very first input, since that allows the game to set things up. Plus they are used to print the welcome message to the player.

Implicit actions are executed by looping over all the actions, handling the ones which has a zero **verb** number and a non-zero **noun** number. The **noun** number is actually a random chance in the range 0 .. 100, where 0 means never do it, and 100 means always do it.

Of course implicit actions must satisfy all their conditions before executing their opcodes. Any number of actions may be executed, and if one action fails it does not affect any other one.

12.4 Handling Input

The interpreter must read a line of input from the keyboard, then break it up into words. Only the first two words are significant, anything remaining on the line can be ignored.

Unlike other parsers, small words like **A** and **THE** are usually not ignored. This is especially important when the word length is only 3, because **THE** could be a valid noun (e.g. **THEATER**).

The result of parsing is a **verb** number and a **noun** number. This first word of the input is looked up in the list of verbs from the vocabulary chunk. The second word (when present) is looked up in the list of nouns. When there is only one word, the noun value will be zero.

Synonyms must also be handled—e.g. **WALK** is usually a synonym for **GO** and should produce the same verb number.

When either the verb or noun was not found, then the interpreter displays a message about not understanding the player's input. Ideally this would not cost the player a turn.

12.5 Movement Commands

The interpreter is **fully** responsible for handling movement in the six standard directions. The story file can only specify the connections between rooms, but has no way to override a movement command, or make movement conditional (for example: whether a door is open or closed). What a shame! The mechanism could have worked just like the **GET** and **DROP** commands (see below).

The interpreter must check for the verb **GO** (always verb #1) followed by a direction noun, or just a direction noun used by itself. When the connection exists, it moves the player into the corresponding room and updates the screen (to display the new room's description). When no connection exists, the move is refused with a suitable message.

Movement when it is dark requires special handling. Dark here means that the **DARK** flag is set and there is no light source nearby. Firstly, the interpreter displays a message about it being dangerous to move in the dark. Secondly, when the move is possible, the player is moved as normal, otherwise a message is displayed that the player has fallen down and died, and the game is completely over!

See the [Hard-coded Messages](#) section in Chapter 8 for more information about what messages to display.

Lastly, the interpreter must check for the abbreviated directions (**N**, **S**, **E**, **W**, **U**, **D**), and convert them to their full form. These abbreviations should work alone or after the **GO** verb.

12.6 Other Commands

Once the **verb** and **noun** numbers are known, and the command is not a movement command, then the action list is scanned to find a matching action. When an action is fully matched (including all conditions being satisfied), then the action is executed and the command has been handled.

If nothing was displayed, and no **look** opcode was executed (including implicitly due to the player changing rooms), then the interpreter must print "O.K." or something similar.

See the [Executing an Action](#) section in Chapter 3 for more information about how actions are matched and executed.

Unmatched Commands

When the **GO** verb is lacking a direction or object, a message about needing a direction is displayed.

When an action failed to match due to a missing noun, then the message "What ?" is displayed.

When at least one action matched the verb and noun, but the conditions did not match, a message like "I can't do that yet" is displayed.

Otherwise there was no match of the verb/noun combination, and a message about not understanding the command is displayed.

12.7 Get and Drop

The **GET** and **DROP** commands require special handling. The interpreter must first scan the action list looking for a match with a verb and noun (as for other commands) and when found, execute that action.

When there was no matching action, the interpreter then looks for the noun in the **item list**, checking each item's vocabulary word against the noun (as text). When found, the interpreter will attempt to pick up or drop the item, performing the obvious checks: the item is in the same room, the player's carry limit is not reached, the player can actually see the item, etc.... If the attempt fails, or the item could not be found, then a suitable message is displayed.

Multiple items may match the noun. When this happens, the interpreter should pick the most likely match — e.g. for **GET** prefer the item which is in the same room, or for **DROP** prefer the item which is being carried.

See the [Hard-coded Messages](#) section in Chapter 8 for more information about what messages to display.

Doing it ALL

The **GET** and **DROP** commands may be followed by the word **ALL**. This word should never be defined in a story file as a noun or item, it is hard-coded in the interpreter.

The interpreter first makes a list of all the items to attempt to pick up or drop. For **GET**, it is all the **gettable** items in the current room (items which have a vocabulary word). For **DROP**, it is simply all items carried by the player.

NOTE : using **GET ALL** when it is too dark to see should produce a single refusal message. A single message is also preferred when the player was already at their carry limit.

Next, each item is attempted to be picked up or dropped in turn, performed as though the player had typed the full command. In particular, a matching action **must** be found and executed.

Before attempting each item, the interpreter should display the name of the item followed by a colon (:) and a space.

After attempting each item, the interpreter should check the game state, and abort when it seems something went wrong, especially if the player has moved to a different room (perhaps because they died and are now in the Limbo room). The player's carry limit should be checked during a **GET ALL** and abort the sequence early.

The whole sequence is considered a single turn — no implicit actions should be run during the sequence.

12.8 Saving the Game

The `save_game` opcode performs all the necessary steps to save the current game. There is no standard for the format of save files, or any convention for their filenames. The original TRS-80 versions and **ScottFree** just used a text file containing numbers separated by whitespace.

The command to save a game was usually two words, `SAVE GAME`. This meant that the verb `SAVE` could safely be used for other purposes.

Notably, there is no opcode to restore a saved game. Some interpreters would ask about restoring a game just before playing it. The **ScottFree** and **ASFD** interpreters would accept a save-file on the command line. Modern interpreters should support a `LOAD GAME` or `RESTORE GAME` command, requiring the second word (as per saving) in case the verbs `LOAD` and `RESTORE` are used for other things in the game.

12.9 The Lamp Timer

For each turn, the lamp timer will count down from its initial value (as given in the header) under the following circumstances:

- item #9 is a valid item
- item #9 has a location which is not `nowhere` (room #0)
- the current value is `> 0`
- the initial value given in the header is not `-1`

When there is less than 25 turns remaining, and the lamp item is held by the player or in the same room, then a message should be displayed to notify the player. Such messages should be displayed even when the `DARK` flag is clear. Early games showed a message every turn, but the **ScottFree** interpreter defaults to showing a message every 5 turns.

Once the timer reaches zero, the interpreter must set the `LIGHT_OUT` flag (number 16), and if the lamp item is nearby then show a message to the player. It is the responsibility of the game code to check this flag and replace the lamp item with its turned-off version.

In the Brian Howarth games, the lamp item is **destroyed** when the lamp counter reaches zero. An interpreter should check if the current game is a Brian Howarth game, e.g. by detecting distinctive messages or room names, and implement this behavior when it is.

Chapter 13: Screen Model

13.1 Split-Screen

This is the traditional model used for by most interpreters, including **ScottFree**. There are two text windows, called the "upper" and "lower" windows. The upper window contains the description of the current room, including a list of exits and a list of items. The lower window is used for player input and displaying messages.

In the TI99/4A games, the upper window usually showed the player's inventory as well.

The number of lines in each window may be fixed. For example, **ScottFree** uses 10 lines for the upper window, and 13 lines for the lower window.

Alternatively, the upper window may resize itself to fully contain the room description. When it becomes larger, it obscures part of the lower window, and when becoming smaller it (ideally) reveals previous text, or simply clears that area. If an increase in size would overlap the cursor in the lower window (i.e. where messages will be printed next), then that cursor should be moved down.

There is usually a line which separates the two windows. In the TRS-80 versions, there was a text line consisting of hyphens with angle brackets at the end, such as: `< ----- >`. For the Commodore-64, a line of asterix characters (*) was common. Some interpreters distinguished the two windows by using different background and/or foreground colors, often with a solid line in-between.

Text in the lower window usually begins at the bottom, and scrolls up as needed. The room description in the upper window is drawn from the top left corner onward, and any unused lines are cleared. When the the upper window prints too much text to fit (and it cannot resize), then the result is implementation-defined.

13.2 Single Window

The basic screen model only contains a single text window, which is used for player input and all text output. Games for the VIC-20 home computer used this model, and the **ASFD** interpreter was created (as a fork of **ScottFree**) to replicate a similar style for later computers.

One difficulty for this screen model is that the **look** opcode was often used to simply update the upper window (in the split-screen model), e.g. after picking up or dropping an object. In that model, it was safe to use this opcode even when not strictly necessary. For the basic model, using this opcode to trigger a redisplay of a room's description can work poorly in some games.

For interpreters using the basic screen model, it is recommended to ignore the `look` opcode and to only display the room's description in the follow circumstances:

- a `LOOK` command (with no noun) was entered by the player
- a successful movement command (like `N` or `GO DOWN`) was entered
- the player's location changed since their last input
- the state of being dark (no nearby light source) changed

13.3 Graphics

Many of a the commercial games supported graphics. There was usually a picture of the current room drawn in an area above the upper window. In some versions, the picture completely replaced the upper text window, though there was often a command to toggle between them. In a few versions, the picture would be displayed upon entering a new room, and once the user pressed a key it was replaced by the normal upper window.

Room pictures were drawn in a few different ways. The games by Brian Howarth used lines and flood-filling. Some games used the text mode of the computer in a clever way, carefully using custom characters to render the scene. These techniques were economical with space, and could be stored in the game binary. Later games used bitmap graphics which were loaded from disk.

Some games also had a screen to show the player's inventory, with small pictures for each item. The game *Buckaroo Banzai* could also draw the items present in the room overlaid onto the room's picture. A few games used the `draw` opcode to show alternate pictures, e.g. when the player examines a gremlin in the game *Gremlins*.

Chapter 14: Known Games

NOTE : this chapter is not attempting to list **all** the games in the world which this specification applies to. Rather it is a list of the most well-known games, especially ones which are readily available to download from the internet.

14.1 Scott Adams

Scott Adams has kindly given permission for his early adventure games to be freely redistributed and enjoyed.

The following games are available in the [adams.zip](#) package at the IF archive website (<https://www.ifarchive.org>) in the directory [scott-adams/games/scottfree](#). The same directory contains several other packages of games.

1	Adventureland
2	Pirate Adventure
3	Secret Mission
4	Voodoo Castle
5	The Count
6	Strange Odyssey
7	Mystery Fun House
8	Pyramid of Doom
9	Ghost Town
10	Savage Island, part I
11	Savage Island, part II
12	The Golden Voyage
13	Sorcerer of Claymorgue Castle
14a	Return to Pirate's Isle
14b	Buckaroo Banzai

Questprobe

The above package also contains two games of the *Questprobe* trilogy, which involve characters from the Marvel™ comics.

1	The Hulk
2	Spiderman

14.3 Brian Howarth

Brian Howarth wrote his eleven "Mysterious Adventures" in 1981-1983, and has also graciously allowed his games to be freely redistributed and enjoyed. The following games are available in the [mysterious.tar.gz](https://www.ifarchive.org) package at the IF archive website (<https://www.ifarchive.org>) in the same directory as mentioned above.

There are also a "blorbified" version of these games which include graphics, however these require an interpreter which can read [.blorb](#) files (and show graphics!).

1	The Golden Baton
2	The Time Machine
3	Arrow of Death, part 1
4	Arrow of Death, part 2
5	Escape from Pulsar 7
6	Circus
7	Feasibility Experiment
8	The Wizard of Akyrz
9	Perseus and Andromeda
10	Ten Little Indians
11	Waxworks

14.4 Other Authors

These eight games can be found in the `otheradv.zip` package at the IF archive site (<https://www.ifarchive.org>) in the same directory as mentioned above.

1	Miner's Adventure	Bruce Hanson
2	Burgler's Adventure	Bruce Hanson
3	James Bond Adventure	?
4	Romulan Adventure	Jim Veneskey
5	Top Secret Adventure	Jim Veneskey
6	Gamma World	Jim Veneskey
7	Marooned	Kim Watt
8	Undersea Conquest	Marc Mandel

14.5 Ghost King

Ghost King is a highly regarded game by Jason Compton, created in 2020. It is playable on-line, and is free to download and play locally. For more information, search for it on IF Database website (<https://ifdb.org>).